

RL Codebase

Technical Reference Report

Exhaustive documentation of `agent.py` · `train.py` · `test_random_policy.py`

Hopper-v4 | REINFORCE & Actor-Critic | PyTorch + Gymnasium

#	Section	Content
1	<code>agent.py</code>	<code>discount_rewards</code> · Policy class · Agent class
2	<code>train.py</code>	<code>main()</code> training loop — full walkthrough
3	<code>test_random_policy.py</code>	<code>main()</code> baseline random-policy runner
4	Data Flow	End-to-end interaction diagram

Section 1 — agent.py

This is the core module. It defines the neural-network policy and the learning agent. All RL logic — action selection, experience storage, and gradient updates — lives here.

FUNCTION

1.1 discount_rewards(r, gamma)

Signature	discount_rewards(r: Tensor, gamma: float) → Tensor	
Purpose	Converts a sequence of per-step rewards into discounted cumulative returns. The return at step t is the sum of all future rewards, each multiplied by γ raised to the power of the number of steps into the future.	
Returns	A 1-D Tensor of the same length as r , where each element is the discounted return from that time-step onward.	

Parameter	Type	Description
r	torch.Tensor (1-D)	Raw per-step rewards collected during one episode.
γ	float $\in (0, 1]$	Discount factor. Values close to 1 weight future rewards nearly as much as immediate ones; smaller values make the agent more myopic.

How it works — reversed accumulation

The function iterates from the **last** time-step back to the first. At each step it applies the recurrence:

$$G_t = r_t + \gamma \cdot G_{t+1}$$

Starting from zero, `running_add` accumulates the discounted sum and each position in `discounted_r` is filled in reverse order. This is more numerically stable than a forward pass.

```
running_add = 0
for t in reversed(range(r.size(-1))):
    running_add = running_add * gamma + r[t]
    discounted_r[t] = running_add
```

CLASS

1.2 class Policy(torch.nn.Module)

Inherits from	torch.nn.Module — standard PyTorch base class for all neural networks.
---------------	--

Role	Implements the combined Actor-Critic network. The actor head outputs a probability distribution over continuous actions; the critic head outputs a scalar state-value estimate.
------	---

METHOD

`__init__(self, state_space, action_space)`

Parameter	Type	Description
state_space	int	Dimensionality of the observation vector (11 for Hopper-v4).
action_space	int	Dimensionality of the action vector (3 for Hopper-v4 — torques).

Defines and registers all learnable components:

Attribute	Type / Value	Purpose
<code>self.hidden</code>	64 (int)	Number of neurons in hidden layers (both actor and critic).
<code>self.tanh</code>	<code>torch.nn.Tanh()</code>	Non-linear activation function shared across all layers. Squashes values to $(-1, 1)$, preventing exploding activations.
<code>self.fc1/2_actor</code>	<code>Linear(state→64, 64→64)</code>	Two hidden layers of the actor sub-network.
<code>self.fc3_actor_mean</code>	<code>Linear(64 → action_space)</code>	Output layer producing the mean of the Gaussian action distribution.
<code>self.sigma_activation</code>	<code>F.softplus</code>	Maps the raw sigma parameter to a strictly positive standard deviation. <code>softplus</code> is smoother than <code>ReLU</code> .
<code>self.sigma</code>	<code>nn.Parameter (shape: [action_space])</code>	Learnable log-scale exploration noise, initialised at 0.5. Shared across all states — a global, state-independent std.
<code>self.fc1/2_critic</code>	<code>Linear(state→64, 64→64)</code>	Two hidden layers of the critic sub-network.
<code>self.fc3_critic_value</code>	<code>Linear(64 → 1)</code>	Output layer producing a scalar value estimate $V(s)$.

METHOD

`init_weights(self)`

Iterates over all `nn.Linear` modules and applies:

Initialisation	Target	Reason
<code>Normal($\mu=0$, $\sigma=1$)</code>	Weight matrices	Breaks symmetry so neurons learn different features.
<code>Zeros</code>	Bias vectors	Biases start neutral; the network learns to shift them.

METHOD

`forward(self, x) → Normal distribution`

Input x	A state vector as a 1-D or 2-D float Tensor.
Returns	A <code>torch.distributions.Normal</code> object parameterised by <code>(action_mean, sigma)</code> . Calling <code>.sample()</code> on it gives an action; <code>.log_prob()</code> gives the log-density needed for the policy gradient.

Actor forward pass:

```
x_actor = tanh( fc1_actor(x) )
x_actor = tanh( fc2_actor(x_actor) )
action_mean = fc3_actor_mean(x_actor) # no activation – unbounded mean
sigma = softplus(self.sigma)          # ensure sigma > 0
return Normal(action_mean, sigma)
```

Note: the **critic forward pass** is intentionally *not* called here — it has its own dedicated method `value(x)` to keep gradients separate.

METHOD**value(self, x) → Tensor (scalar per state)**

Input x	State vector(s) as a float Tensor.
Returns	A scalar (or batch of scalars) $V(s)$ — the critic's estimate of expected future return from state s .
squeeze(-1)	Removes the trailing size-1 dimension from <code>fc3_critic_value</code> so the output shape matches the rewards Tensor for element-wise MSE.

CLASS**1.3 class Agent**

Role	Wraps the Policy. Manages the experience buffer, runs the optimiser, and implements both learning algorithms (REINFORCE and Actor-Critic).
-------------	--

METHOD**__init__(self, policy, device, algorithm)**

Attribute	Default	Purpose
<code>self.train_device</code>	"cpu"	torch device where tensors and the model live.
<code>self.policy</code>	Policy instance	The neural network, moved to <code>train_device</code> .
<code>self.optimizer</code>	Adam, <code>lr=1e-3</code>	Gradient-based optimiser. Adam adapts learning rates per parameter.
<code>self.algorithm</code>	"reinforce"	Selects the loss computation branch in <code>update_policy</code> .
<code>self.gamma</code>	0.99	Discount factor used in return / TD-target computation.

<code>self.states</code>	<code>[]</code>	Buffer of state tensors seen during the current episode.
<code>self.next_states</code>	<code>[]</code>	Buffer of next-state tensors (needed only for Actor-Critic TD targets).
<code>self.action_log_probs</code>	<code>[]</code>	Buffer of $\log \pi(a s)$ for each step — used to compute the policy gradient.
<code>self.rewards</code>	<code>[]</code>	Buffer of per-step scalar rewards.
<code>self.done</code>	<code>[]</code>	Buffer of episode-termination booleans (masks TD target at terminal states).

METHOD

get_action(self, state, evaluation=False)

Purpose	Converts a raw numpy state into an action. Handles the train/eval distinction.	
Input	state: numpy array from <code>env.step()</code> or <code>env.reset()</code> .	
Returns	(action Tensor, log_prob Tensor) — or (mean Tensor, None) in eval mode.	
Branch	Condition	Behaviour
Evaluation	<code>evaluation=True</code>	Returns <code>Normal.mean</code> deterministically — no randomness, best known action.
Training	<code>evaluation=False</code>	Samples from Normal distribution (exploration). Computes $\log \pi(a s)$ = sum of per-dimension log-probs.

The **sum** of per-dimension log-probs is correct because the multivariate action distribution is factorised as a product of independent 1-D Gaussians: $\log p(a) = \sum \log p(a_i)$.

METHOD

store_outcome(self, state, next_state, action_log_prob, reward, done)

Appends one time-step of experience to the five buffers. States are converted from numpy to float32 Tensors immediately; the reward is wrapped in a 1-element Tensor for later stacking.

METHOD

update_policy(self)

Called once per episode. Converts buffers to stacked Tensors, clears them, then branches on `self.algorithm`.

REINFORCE branch	
Step 1	Compute discounted returns <code>G</code> via <code>discount_rewards(rewards, gamma)</code> .
Step 2	Normalise returns: subtract mean, divide by <code>std + ε</code> . Reduces variance of gradient estimates.

Step 3	Loss = $-\text{mean}(\log \pi(a s) \cdot G)$. Maximising expected return $\equiv$ minimising this loss.
---------------	--

ACTOR-CRITIC branch

Step 1	values = $V(s)$ from critic. next_values = $V(s')$ (detached — no gradient through target).
Step 2	targets = $r + \gamma \cdot V(s') \cdot (1 - \text{done})$. The $(1 - \text{done})$ mask zeroes out bootstrapped value at terminal states.
Step 3	advantages = targets – values. Positive advantage ■ action was better than expected.
Step 4	actor_loss = $-\text{mean}(\log \pi(a s) \cdot \text{advantage.detach}())$. Detach prevents advantage from being differentiated through the critic.
Step 5	critic_loss = $\text{MSE}(\text{values}, \text{targets})$. Trains the value estimator to be accurate.
Step 6	loss = actor_loss + critic_loss. Single backward pass updates both heads.

After computing the loss, all three algorithms follow the same optimiser steps:

```
optimizer.zero_grad() # clear old gradients
loss.backward()       # compute new gradients via autograd
optimizer.step()      # update all parameters
```

Section 2 — train.py

This is the entry point for training. It sets up the environment, instantiates the agent, and runs the main episode loop.

FUNCTION

2.1 main()

All logic is contained in this single function. Below are its components in order.

Hyperparameters / Configuration

Variable	Value	Meaning
n_episodes	5000	Total number of training episodes.
log_interval	10	Print a performance summary every N episodes.
seed	0	Base random seed. Each episode uses seed+episode for reproducibility while still varying resets.
device	auto	"cuda" if a GPU is available, else "cpu". Tensors and model are placed here.

Environment Setup

<code>gym.make(...)</code>	Creates the MuJoCo Hopper-v4 physics simulation. <code>render_mode="human"</code> opens a real-time viewer window.
<code>env.action_space.seed</code>	Seeds the action-space RNG for reproducibility of random elements.
<code>observation_space</code>	<code>Box(11,)</code> — the 11-dimensional state: joint positions, velocities, etc.
<code>action_space</code>	<code>Box(3,) ∈ [-1, 1]³</code> — torques applied to ankle, knee, and hip joints.

Agent / Policy Instantiation

```
policy = Policy(env.observation_space.shape[0], env.action_space.shape[0])
agent = Agent(policy, device=device, algorithm="actor_critic")
```

Switches the algorithm between "reinforce" and "actor_critic" on this line.

Training Variables

Variable	Initial value	Purpose
best_return	$-\infty$	Tracks the highest episode return ever achieved.
running_return	0.0	Exponential moving average ($\alpha=0.05$) of episode returns — a smoothed performance signal.

Episode Loop (for episode in range(1, n_episodes+1))

Step	Code / Action	Explanation
Reset	<code>state, _ = env.reset(seed=...)</code>	Resets the simulation to a random initial posture. Seed is varied per episode.
Act	<code>action, log_prob = agent.get_action(state)</code>	Forward pass through the actor to sample an action.
Clip	<code>action_np.clip(low, high)</code>	Ensures the sampled action stays within the physical joint limits.
Step	<code>env.step(action_np)</code>	Advances the simulation by one timestep; returns <code>next_state</code> , <code>reward</code> , <code>terminated</code> , <code>truncated</code> .
done flag	<code>terminated or truncated</code>	<code>terminated</code> = hopper fell; <code>truncated</code> = max episode length reached (1000 steps by default).
Store	<code>agent.store_outcome(...)</code>	Appends the transition to the experience buffers.
Return	<code>episode_return += reward</code>	Accumulates the undiscounted sum of rewards for logging.
Update	<code>agent.update_policy()</code>	After the episode, perform one gradient update.
EMA	<code>0.95 * running + 0.05 * episode</code>	Smooth the return signal to track long-term trend.
Log	<code>print(...)</code>	Every <code>log_interval</code> episodes, print episode / running / best returns.

Section 3 — test_random_policy.py

A minimal diagnostic script. Its only purpose is to run the Hopper-v4 environment with completely **random actions** — no learning involved. This establishes a lower-bound performance baseline and verifies the environment works.

FUNCTION

3.1 main()

Variable	Value	Purpose
render	True	If True, opens a graphical viewer. Set to False for headless runs.
n_episodes	50000	Very large number — effectively "run until killed".
a	1	One-time flag: env.reset() is called only on the very first episode, then done is relied upon to implicitly continue. Note: this is a minor bug — done=True is never used to reset, so the environment keeps stepping even after termination.

Inner loop

<code>env.action_space.sample()</code>	Draws a uniformly random action from $[-1, 1]^3$. No policy, no learning.
<code>env.step(action)</code>	Steps the simulation. Returns state, reward, terminated, truncated, info.
done flag	terminated or truncated — exits the while loop. However due to the <code>a=0</code> bug, <code>env.reset()</code> is never called again after episode 0, so subsequent "episodes" just continue from wherever the prior one ended.
<code>env.render()</code>	Renders the current frame to the window (only called if <code>render=True</code>).

Known issue — missing reset

The variable `a` is set to 0 after the first reset and never reset to 1. Combined with the fact that the outer loop does not call `env.reset()` at the start of each episode, the environment is effectively **never reset after episode 0**. This does not affect baseline measurements much (random policy performance is near-zero regardless), but it is a bug that should be corrected for any rigorous baseline evaluation.

Section 4 — End-to-End Data Flow

The table below traces the exact data transformations from raw environment observation to policy gradient update, step by step.

#	Producer	Data / Tensor	Consumer	Shape (Hopper)
1	env.reset() / env.step()	state (numpy)	agent.get_action()	(11,)
2	Policy.forward()	Normal(mean, sigma)	Normal.sample()	—
3	Normal.sample()	action Tensor	env.step()	(3,)
4	Normal.log_prob()	$\log \pi(a s)$ scalar	store_outcome()	()
5	env.step()	next_state, reward, done	store_outcome()	(11,), (), ()
6	store_outcome()	episode buffers	update_policy()	$T \times (11,)$ etc.
7	discount_rewards()	returns G_t	policy loss	(T,)
8	Policy.value()	$V(s), V(s')$	advantage A_t	(T,)
9	advantage	$A_t = \text{target} - V(s)$	actor_loss	(T,)
10	MSE($V(s)$, target)	critic_loss scalar	total loss	()
11	loss.backward()	gradients $\nabla \theta$	Adam.step()	all params

Algorithm Comparison

Property	REINFORCE	Actor-Critic
Return estimate	Full episode G_t (Monte Carlo)	One-step TD target $r + \gamma V(s')$
Variance	High — waits for full return	Lower — critic provides baseline
Bias	Zero (unbiased)	Some bias from critic approximation
Requires critic	No	Yes
Update frequency	Once per episode	Once per episode (same code here)
Typical use	Simple / short episodes	Complex / long-horizon tasks

Key PyTorch / RL Concepts — Quick Reference

Term	Definition
<code>torch.nn.Module</code>	Base class for all neural networks. Tracks parameters and provides <code>.to(device)</code> , <code>.parameters()</code> , etc.
<code>torch.distributions.Normal</code>	Parameterises a Gaussian. <code>.sample()</code> gives reparameterised samples; <code>.log_prob()</code> computes log-density.
<code>F.softplus(x)</code>	$\log(1 + e^x)$ — smooth approximation of ReLU, guarantees positive output for σ .

<code>.detach()</code>	Stops gradient flow through a tensor. Used for TD targets and advantages to avoid second-order gradients.
Adam optimiser	Adaptive Moment Estimation. Maintains per-parameter running mean and variance of gradients; robust choice for RL.
Advantage $A(s, a)$	$A(s, a) = Q(s, a) - V(s)$. Tells the actor whether an action was better or worse than average. Reduces variance vs. raw returns.
done mask ($1 - \text{done}$)	Zeroes the bootstrap term $V(s')$ at terminal states, preventing the agent from expecting future rewards after the episode ends.